Extra Exercise

Lecture 06: Build a BLE System and Demonstrate It

Communication Technologies for the Internet of Things

Academic year 2026/2027

Catarina Silva

ESTGA, University of Aveiro

Overview

The goal of this exercise is to bring everything together in a system that works end to end and, above all, to use it to *observe and measure* how BLE behaves.

The system is a room monitoring station. A node made of an Arduino and an HM-11 module measures temperature and humidity and is commanded over BLE. A Raspberry Pi connects to the node, receives the readings, logs them and shows them on a dashboard. What is assessed at the end is not the dashboard itself, but what the group can demonstrate and explain about the communication technology behind it.

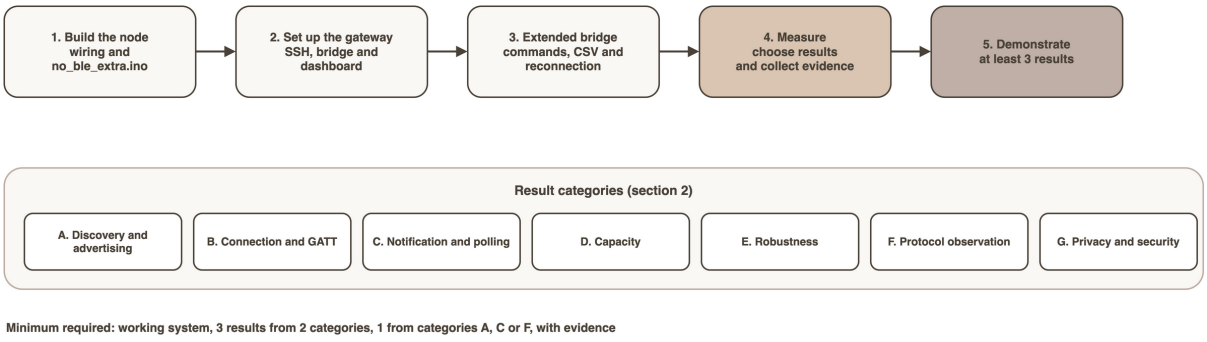


Figure 1: *Stages of the exercise*

Materials

- Raspberry Pi with a prepared card, SSH enabled, connected to the lab network, with BlueZ, bluepy and Node-RED with the `@flowfuse/node-red-dashboard` package.
- Personal computer, for SSH access and to view the dashboard.
- Arduino with an HM-11 shield. The DHT11 and the LED, connected to pin 8 through a 220 Ω resistor, are optional (subsection "Without all the sensors").
- Smartphone with the *nRF Connect* app (or equivalent), as a second client.
- The `codigo/` folder, with `ponte_ble_tcp.py` from Lecture 05, plus `no_ble_extra.ino`, `ponte_estendida.py`, `ponte_at.ino`, `varredor.py` and `medidor_ble.py`.

Each group needs one node, made of an Arduino and an HM-11, and one Raspberry Pi. If either is missing, two groups share the equipment and take turns, because the HM-11 accepts one client at a time. While one group measures, the other prepares the dashboard, reads the code or organises the evidence.

Learning Objectives

- Integrate peripheral, gateway and visualisation into a single working BLE system;
- Measure quantities that characterise a BLE link (RSSI, latency, cadence, losses);
- Relate what is observed to the protocol parameters (advertising interval, connection interval, *supervision timeout*, MTU);
- Support every claim with evidence: a number, a chart or a capture.

1. System to Build

Arduino Wiring

The HM-11 shield is connected as in Lecture 05. The DHT11 and the LED are added on the pins shown. The LED must only be connected with the series resistor.

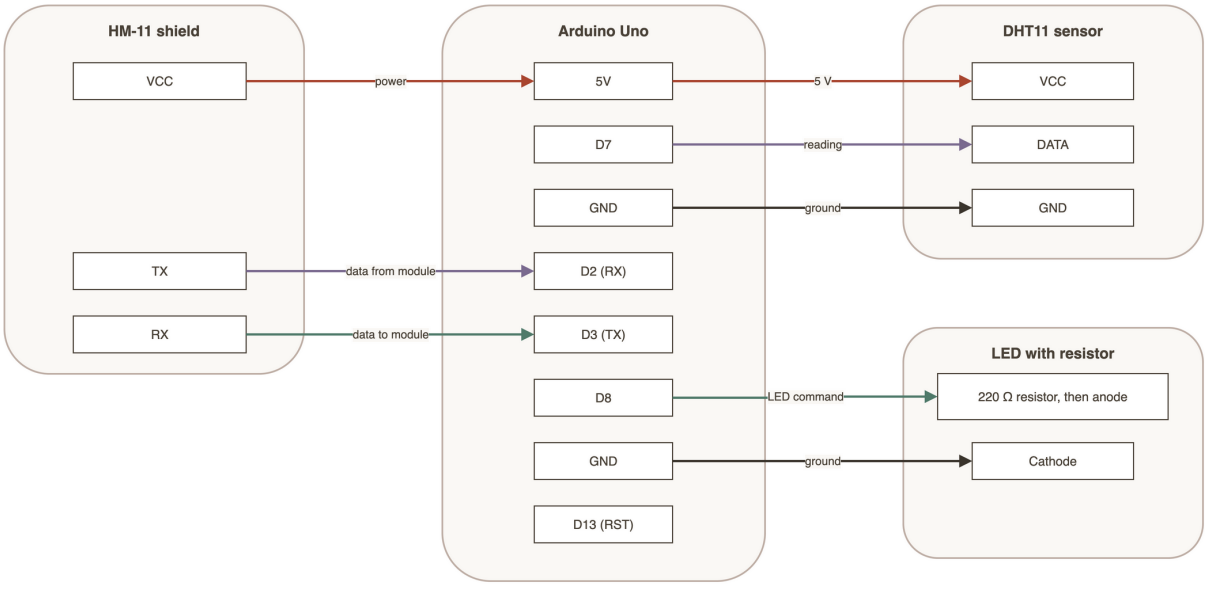


Figure 2: *BLE node wiring*

Architecture

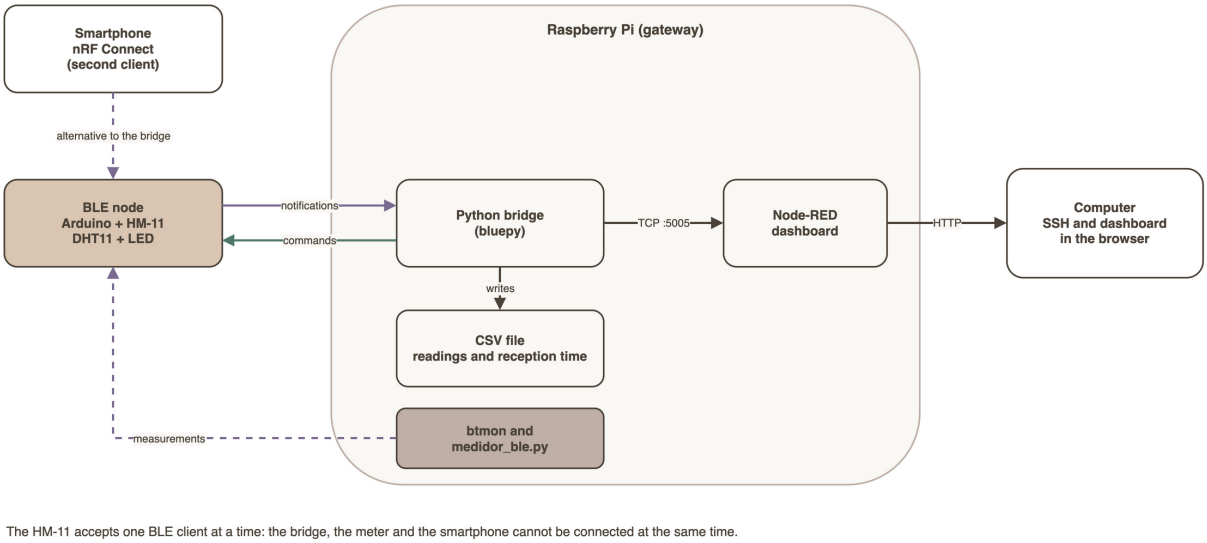


Figure 3: System architecture and what runs on each element

Accessing the Raspberry Pi

In a real installation the gateway sits where the sensors are, without a monitor or keyboard. Work is therefore done over SSH from the personal computer, which reproduces that condition and avoids one monitor per workstation.

1. Get the address assigned to the Raspberry Pi on the lab network.
2. Open the remote session and note the operating system version.
3. Check that the Bluetooth adapter exists and bring it up.

```
ssh pi@<RASPBERRY_ADDRESS>
cat /etc/os-release
hciconfig
sudo hciconfig hci0 up
```

Discovering the Node

With the node powered on, run the scanner. It lists the BLE devices advertising in the neighbourhood, with their address, address type, RSSI and name, and it is the way to obtain the node address.

```
sudo python3 codigo/varredor.py 15
```

The output is richer than that of the `hcitool lescan` command from Lecture 04, because it includes the RSSI and the name extracted from the advertising data. The RSSI is shown in dBm and is always negative. It orders devices by relative proximity, but it is not a distance measurement.

The Bridge

The bridge connects to the node, enables notifications by writing to the characteristic's configuration descriptor, and forwards each notification received to `127.0.0.1:5005`, from where Node-RED consumes it.

```
sudo python3 codigo/ponte_ble_tcp.py <NODE_ADDRESS>
```

Node-RED Dashboard

1. Drag a `tcp in` node, in *Connect to* mode, to `127.0.0.1:5005`, delivering *strings* delimited by `\n`.

2. Drag a `function` node with two outputs and paste the following content, which reads positions 1 and 2 because position 0 is the counter:

```
var v = msg.payload.split(",");
return [ { payload: parseFloat(v[1]) },
         { payload: parseFloat(v[2]) } ];
```

3. Connect the first output to a `ui-chart` named *Temperature* and the second to a `ui-gauge` named *Relative humidity*.

4. Press `Deploy` and open the dashboard.

Phases

The work is done in three phases, each building on the previous one. Only the first phase has to work for the demonstration.

1. In the first phase the node runs `no_ble_extra.ino`, the bridge from Lecture 05 receives the notifications and the Node-RED dashboard shows at least one chart.

2. In the second phase the Lecture 05 bridge is replaced by the extended bridge, which can send commands to the node, for example to switch the LED on and off, and writes each reading to a CSV file with its reception time.

3. In the third phase the automatic reconnection of the extended bridge is tested, by powering the node off and on again, and a connection status indicator is added to the dashboard.

The Extended Bridge

The extended bridge is provided ready to run. The work consists in running it, reading it and adapting it, for example to change the sending period, change the message format or add a field to the file.

```
sudo python3 codigo/ponte_estendida.py <NODE_ADDRESS> leituras.csv
sudo python3 codigo/ponte_estendida.py <NODE_ADDRESS> leituras.csv --
   sondagem
```

Each line typed in the terminal is sent to the node as a command. The file records the time of each message and of each connection event, with the cumulative number of lost messages, obtained from the node counter. If the connection drops, the bridge retries with increasing waits, enables notifications again and resumes sending. The `-sondagem` option requests one reading per second instead of using the node's spontaneous sending, which allows the two modes to be compared. The connection status indicator on the dashboard can rely on a `function` node that flags the absence of readings during a given interval.

Node commands

Commands are single characters written to the data characteristic. The characters `t`, `h` and `X` request a reading. The characters `1` and `0` switch the LED on and off. The character `S` enables spontaneous sending and `P` disables it. The characters `a` to `e` select the period of that sending: 1, 2, 5, 10 or 30 s. In `X` mode each message has the form `counter,temperature,humidity`, which makes it possible to detect lost messages. In Lecture 05 the message only had temperature and humidity, so the dashboard `function` node, which read positions 0 and 1, now reads positions 1 and 2.

Module configuration

Some results in the next section require changing the name, the advertising interval or the transmit power of the HM-11. To do so, upload `ponte_at.ino`, open the Serial Monitor at 9600 baud with no line ending and send `AT` commands, for example `AT+NAME?`, `AT+ADVI?` and `AT+POWE?`. These commands are only accepted when the module has no active connection. After configuring, upload `no_ble_extra.ino` again.

2. Results to Demonstrate

With the system running, each group chooses the results it wants to demonstrate. Each row of the following tables describes a possible result and the evidence that supports it. The level indicates the relative effort: 1 is a direct observation, 2 requires a repeated measurement and 3 requires a measurement, a capture or a change to the system.

A. Discovery and Advertising

Id	Result to demonstrate	Evidence	Level
A1	The node advertises with a name chosen by the group and that name appears on the gateway and on the smartphone.	Scanner output and smartphone screen.	1
A2	Decoding of the advertising data: *flags*, name, transmit power and any manufacturer data.	Table with each field, its AD type and its value.	2
A3	The RSSI decreases with distance, but with large variability.	RSSI chart at four or more distances, 30 samples each, with mean and standard deviation.	2
A4	Effect of obstacles (body, wall, metal drawer) on the RSSI at the same distance.	Comparison table and a conclusion on how useful the RSSI is as a distance measure.	2
A5	The advertising interval determines how many times the node is detected in a scan.	Detection count in 10 s for at least three values of `AT+ADVI`.	3
A6	The transmit power changes the RSSI and the range.	RSSI at the same position for at least two values of `AT+POWE`.	3

B. Connection and GATT

Id	Result to demonstrate	Evidence	Level
B1	The node's GATT profile: services, characteristics, properties and *handles*.	Listing obtained by program and confirmed on the smartphone.	1
B2	The configuration descriptor sits at the handle after the characteristic's, and without writing to it no notifications arrive.	Run without the write, run with the write, and an explanation of the value written.	2
B3	Writing `0x0000` to the descriptor stops notifications without ending the connection.	Log of the stop and of the resumption.	2
B4	Write with and without response.	Difference observed in the code and in a capture of each.	3

C. Notification versus Polling

Id	Result to demonstrate	Evidence	Level
C1	Spontaneous sending generates less traffic than periodic polling for the same cadence.	Number of ATT PDUs in 60 s in both modes, counted on a capture.	3
C2	Latency of a command: time between the write and the arrival of the response.	Histogram or table of at least 50 samples, with median and maximum.	2
C3	Actual cadence of notifications against the requested period.	Intervals between consecutive messages for at least two periods.	2
C4	Command from the gateway to the node: the LED lights up and the confirmation returns to the dashboard.	Live demonstration and measured time.	1

D. Capacity

Id	Result to demonstrate	Evidence	Level
D1	A message longer than 20 octets arrives fragmented in several notifications.	Two consecutive notifications visible on a capture and the relation to the 23 octet MTU.	3
D2	Maximum useful throughput achieved between the gateway and the node.	Useful octets per second, with the method and the limiting factor.	3
D3	Header overhead: useful octets against total octets of a notification.	Calculation based on a captured notification.	2

E. Robustness

Id	Result to demonstrate	Evidence	Level
E1	Time the gateway takes to declare the connection lost when the node loses power.	Measurement repeated at least five times and relation to the *supervision timeout*.	2
E2	Automatic recovery: the gateway reconnects by itself and the dashboard resumes.	Log with the time of the loss, the time of the resumption and the number of messages lost according to the counter.	2
E3	While the gateway is connected, the node is no longer visible and a second client cannot connect.	Smartphone scan before and after, and an explanation.	1
E4	Effect of interference in the 2.4 GHz band (Wi-Fi, microwave oven, human body).	Losses or RSSI with and without the interference source.	3

F. Protocol Observation

Id	Result to demonstrate	Evidence	Level
F1	The complete sequence of a session: advertising, connection request, MTU exchange, service discovery, descriptor write, notifications.	Annotated capture with `sudo btmon`, one stage per line.	3
F2	The hexadecimal content of a notification matches the text sent by the Arduino.	Captured octets and their conversion.	1
F3	Negotiated connection parameters: interval, latency and *supervision timeout*.	Values read from the capture and comparison with what the theory predicts.	2

G. Privacy and Security

Id	Result to demonstrate	Evidence	Level
G1	Information collected about the devices in the room without any authorisation.	Table with address, address type (public or random) and name, and a discussion.	1
G2	Any client that connects to the node can read the data and command the LED.	Smartphone switching the LED on without pairing, and a mitigation proposal.	2

3. Demonstration

At the end of the session each group shows the system running and presents the chosen results.

1. Show the system running, at least in the first phase.
2. Choose at least three results, from at least two distinct categories, of which at least one is from categories A, C or F.
3. Prepare the evidence for each result, ready to show without new measurements.

Extra

What to Do, Step by Step

1. Assemble the Arduino node according to the wiring figure and upload `no_ble_extra.ino`.
2. Access the Raspberry Pi over SSH and check that the Bluetooth adapter is operational (subsection “Accessing the Raspberry Pi”).
3. Run the scanner, identify the node address and note the initial RSSI.
4. Run `sudo python3 codigo/ponte_ble_tcp.py <ADDRESS>` and check in the terminal that notifications arrive.
5. Build the Node-RED dashboard with a chart and confirm that the first phase works.
6. Switch to the extended bridge and test the commands, the CSV file and the reconnection, adapting it to the group.
7. Add a connection status indicator to the dashboard.
8. Go through the tables in section 2 and choose the results to demonstrate.
9. Collect the measurements and the evidence for each chosen result, with `medidor_ble.py`, `btmon` and the smartphone.

10. Prepare the demonstration and present it to the instructor.

Without All the Sensors

The exercise needs the HM-11 connected to the Arduino and nothing else. All the results in section 2 concern the BLE link, and none depends on the DHT11 or the LED.

At the top of `no_ble_extra.ino` there are two constants. With `USAR_SENSOR` set to 0, the readings are simulated by a random walk around 22 °C and 45 %, without a DHT11 or its library. With `USAR_LED` set to 0, the `1/0` command acts on no pin and is confirmed on the Arduino Serial Monitor. Pin 13 cannot be used as an alternative LED, because it drives the HM-11 reset. The simulated values keep the `counter,temperature,humidity` format, so the dashboard, the file and the measurements work unchanged.

Alternative Sensors

If other sensors are available, any sensor or actuator that the Arduino can read or command will do, as long as the readings are sent along the same path. The change is made in the sketch function `enviar()`, which reads the sensor and writes the message to the HM-11 serial port. Pins D2 and D3 are taken by the module and pin D13 by its reset. Possible examples are a DHT22, an analogue temperature sensor, an LDR for light, an ultrasonic distance sensor, a PIR motion sensor or a button, and, as an actuator, a buzzer or a relay module instead of the LED.

If the measured quantity changes, the dashboard and the log file must change accordingly. The results in the tables of section 2 still apply, because they concern the BLE link and not the sensor.

Theoretical Background

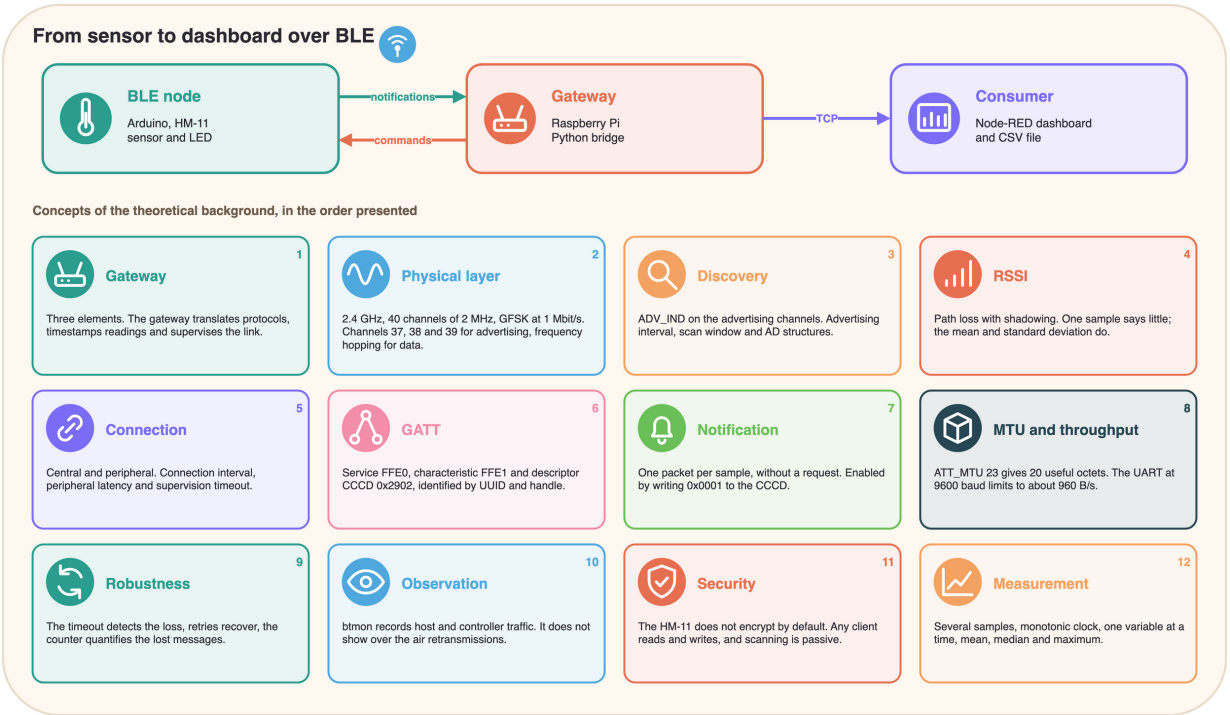


Figure 4: *Graphical summary of the theoretical background*

This section describes systematically what the system built does and why it behaves as it does. Each result in section 2 corresponds to a protocol property described here.

Architecture and Role of the Gateway

The system follows the three element model usual in the Internet of Things. The acquisition node connects to the physical world, measures and acts, and has minimal processing, memory and energy resources. The gateway interconnects two networks of different nature, in this case the BLE link to the node and the local IP network. The consumer, represented by the dashboard and the log file, does not know BLE: it only receives data that has already been translated.

The gateway performs four functions that the node cannot take on. It translates protocols, converting GATT notifications into text messages over TCP. It assigns a reception time to the readings, because the node has no real time clock. It stores and logs, which decouples the node's pace from the consumer's pace. It supervises the connection and decides when and how to restore it.

Physical Layer and Medium Access

BLE operates in the 2.4 GHz ISM band, divided into 40 channels spaced 2 MHz apart, at a symbol rate of 1 Mbit/s with GFSK modulation. Three of these channels, 37, 38 and 39, are reserved for advertising and chosen to avoid the channels most used by Wi-Fi. The remaining 37 channels carry connection data, with frequency hopping between connection events. Frequency hopping is BLE's main defence against interference, and it explains why a narrowband interference source degrades performance without interrupting it.

Discovery: Advertising and Scanning

A peripheral that accepts connections periodically emits *ADV_IND* advertising packets on the three advertising channels. The interval between advertisements, the *advertising interval*, ranges from 20 ms to 10.24 s, in multiples of 0.625 ms, plus a small random delay that reduces collisions. A short interval speeds up discovery and increases consumption; a long interval does the opposite.

An observer in scanning mode listens on each channel during a window, the *scan window*, repeated with a period, the *scan interval*. Detecting an advertisement depends on the coincidence between the instant and channel of the emission and the listening window. The number of detections per unit of time therefore grows with the advertising frequency and with the ratio between the scan window and the scan interval.

Advertising data takes the form of consecutive AD structures, each with one length octet, one type octet and the data. The most common types are the *flags*, the device name, the transmit power and manufacturer data. The device address may be public, assigned by the manufacturer, or random; random addresses can change periodically to make it harder to track a person through their device.

RSSI and Propagation

The *Received Signal Strength Indicator* is the power measured by the receiver, in dBm. In a path loss model with shadowing, the RSSI at distance d is

$$\mathrm{RSSI}(d) = \mathrm{RSSI}(d_0) - 10\,n \log_{10}\left(\frac{d}{d_0}\right) + X_\sigma,$$

where d_0 is a reference distance, n the path loss exponent and X_σ a random variable with zero mean and standard deviation σ. The exponent is close to 2 in free space and typically between 2 and 4 inside buildings. The random term summarises obstacles, antenna orientation, reflections and the presence of people. It follows that a single RSSI reading says little about distance, and only the mean of several samples, together with the standard deviation, is statistically meaningful.

Connection

To establish a connection, the gateway, in the *central* role, answers an advertisement with a connection request. The node becomes the *peripheral* and stops advertising. That is why, while the gateway is connected, the node is no longer visible to other devices and does not accept a second client.

The connection is organised in periodic events in which central and peripheral exchange packets. Three parameters govern its behaviour. The *connection interval*, between 7.5 ms and 4 s, is the period between events. The *peripheral latency*, between 0 and 499, is the number of events the peripheral may skip to save energy. The *supervision timeout*, between 100 ms and 32 s, is the maximum time without valid packets after which the connection is declared lost. These values are proposed by the central and accepted or adjusted by the peripheral, so they need not match those in the module's documentation.

Attributes and GATT

The *Attribute Protocol* (ATT) defines a server that exposes a table of attributes, each with a 16 bit *handle*, a type identified by a UUID and a value. The *Generic Attribute Profile* (GATT) structures that table as a hierarchy of services, characteristics and descriptors. A characteristic groups a value and the properties that say how it can be used, for example read, write, write without response and notify.

The HM-11 module exposes a service with UUID `FFE0` with a single characteristic `FFE1`, which behaves as a transparent serial port. The octets written by the client leave through the module's serial port, and the octets the Arduino writes to that port become the characteristic's value. *Handles* depend on the implementation, so the code identifies characteristics by UUID and derives the descriptor from the characteristic's *handle*.

Notifications and Polling

There are two ways for a client to obtain data from a characteristic. In polling, the client sends a read request and receives a response, which generates two ATT packets per sample and ties the latency to the polling period. In notification, the server sends a *Handle Value Notification* packet

whenever the value changes, without a prior request, which generates one packet per sample and reduces latency to the transmission time.

The client enables notifications by writing the value 0x0001 to the *Client Characteristic Configuration Descriptor* (CCCD, UUID 0x2902), and disables them by writing 0x0000. Without that write the server sends nothing, even if the value changes. A notification is not acknowledged at the ATT level, unlike an indication. Reliability is guaranteed below, by the link layer, which retransmits unacknowledged packets until the end of the *supervision timeout*.

MTU, Fragmentation and Throughput

The default *ATT_MTU* is 23 octets. After the 3 octet ATT header, each notification carries at most 20 octets of useful data. A longer message is split into several consecutive notifications, and it is up to the receiver to put them back together, for example using an end of line delimiter.

Maximum useful throughput is limited by several factors in series. On the BLE side it depends on the number of packets per connection event, the *connection interval* and the 20 octets per packet. On the Arduino side it depends on the module's serial port, which at 9600 bit/s transfers about 960 octets per second. The observed throughput is the minimum of these limits, which identifies the system bottleneck. In terms of efficiency, a notification of 20 useful octets takes up 27 octets of link layer *payload*, because of the L2CAP header, with 4 octets, and the ATT header, with 3, plus 10 octets of preamble, access address, header and CRC. Header overhead is therefore significant for short messages, such as sensor readings.

Robustness and Recovery

Loss of connection is detected by the *supervision timeout*: the central only declares the connection terminated after this time has elapsed without receiving packets from the peripheral. There is a practical consequence: the time the gateway takes to realise that the node has disappeared is of the order of magnitude of that parameter, and is not immediate.

Recovery requires two design decisions. The first is the retry strategy, in which an increasing interval between attempts avoids saturating the adapter when the node is unavailable. The second is the detection of losses during the unavailability, which is solved by numbering the messages at the node. Since the counter is monotonic, a gap in the received sequence exactly quantifies the lost messages.

Protocol Observation

The `btmon` tool records the traffic on the interface between the *host*, which runs on the Raspberry Pi, and the Bluetooth controller. It therefore shows the advertising events received, the connection request and completion with its parameters, the MTU exchanges, service discovery, the CCCD write and each notification with its content in hexadecimal. It does not show what happens over the air, such as link layer retransmissions, which are handled inside the controller. Capturing the air interface requires a dedicated *sniffer*, which is not available in this exercise.

Security and Privacy

BLE provides pairing, link encryption and authentication, but the HM-11 uses an unprotected connection by default. Any client within range can connect, read the data and write to the characteristic, which includes commanding the LED. Scanning is passive and requires no authorisation, so the list of devices in a room, with addresses and names, is collected by any observer. Random addresses that change over time reduce tracking, but only devices that use them benefit from that protection.

Measurement Methodology

The results of the exercise are measurements, and their value depends on the method. A measurement must state what was measured, under what conditions, with what instrument and with how many repetitions.

For random quantities such as RSSI and latency, a single sample is not enough. Collect enough samples to estimate the mean and standard deviation, or the median and percentiles when the distribution is skewed, as is almost always the case with latencies. The maximum value matters because in a real time system it is the tail of the distribution that determines worst case behaviour.

Time intervals are measured with a monotonic clock, which is not affected by jumps in the system clock. The latency of a command is the difference between the instant the write is issued and the instant the response arrives, and therefore includes the wait for the next connection event, the processing on the Arduino and the transmission through the module's serial port. Change one variable at a time, keeping position, orientation and environment constant, so that the observed difference can be attributed to that variable.

Relation to the Results in Section 2

The result categories correspond to the topics above. Category A follows from advertising, scanning and the propagation model. Category B follows from the GATT structure and the CCCD. Category C compares the polling model with the notification model. Category D follows from the MTU, fragmentation and throughput limits. Category E follows from the *supervision timeout*, exclusive connection and the recovery strategy. Category F checks the same concepts in the traffic capture. Category G follows from the security and privacy properties of the protocol.

Discussion Questions

1. What functions does the gateway perform that the peripheral could not, and why?
2. If there were ten nodes instead of one, what changes would the bridge need?
3. Why would the local TCP connection used in this exercise not work in an installation spread across several buildings?
4. What information about the devices in the room could be collected without any prior authorisation?

5. How long does it take for the bridge to declare the connection lost when the node loses power, and which connection parameter does that value relate to?

References

- bluepy library documentation: `https://ianharvey.github.io/bluepy-doc/`
- Raspberry Pi Foundation. Headless setup and SSH access: `https://www.raspberrypi.com/documentation/`
- Node-RED documentation: `https://nodered.org/docs/user-guide/`
- BlueZ and `btmon` documentation: `http://www.bluez.org/`
- HM-11 module datasheet, with the AT command list: `http://www.jnhuamao.cn/`